THE CARTRIDGE API

bm game API, volume 3: 3D

Models, cameras, lights, skeletons, the GPU and collision worlds: everything a game needs to draw and move things in 3D.

Contents

How 3D works in bm **4**

Meshes and models 4

Who draws: the GPU or the ARM 4

Limits you must know 4

The order inside a frame 4

Tutorial: a model on the screen **6**

Meshes and models **7**

Baked light 7

Drawing and material bits **8**

The flags of draw3d 8

Material bits 8

Camera, light and fog **10**

Skeletons and animation **11**

Effects, labels and 2D over the 3D **12**

Visibility **13**

The GPU: gpu3d and its options **14**

Impostors 14

What the GPU also does 15

Anti-aliasing 15

A frame later 15

Collision worlds **16**

Light for 2D scenes **17**

Light 17

Light by levels 17

The n8 library 18

Performance tips **19**

Open points **20**

README

What this is. Volume 3 of the bm game API. It covers every 3D function a cartridge can call, and the lights and collision worlds that go with them.

How to use it. Read the first chapter once, follow the tutorial to get a model on the screen, then keep the other chapters as a reference. Each table gives the function with its arguments and what it does.

What is missing. Every gap is marked with a highlighted TO FILL tag; nothing is invented.

How 3D works in bm

This chapter tells you what happens between your `_draw` and the pixels on the screen.

Meshes and models

A 3D object is a **mesh**: a list of vertices and of triangular faces. You get a mesh in three ways.

- You build it from tables of numbers with `mesh()`.
- You ask for a ready-made one with `mesh_sphere()` or `mesh_cube()`.
- You load a **model of the cartridge** with `model()`. Models are made with bm Studio (MESH section) and carry the cartridge's sprite sheet as their texture. A model with a skeleton comes from bm Animator and can be animated with `animate()`.

You draw a mesh with `draw3d()`, from a camera you place with `camera3d()`.

Who draws: the GPU or the ARM

The Pi's GPU (V3D) draws the triangles. With the setting *Settings > Graphics > 3D of the games* on `ARM` (`gpu3d=0` in `bm/config.txt`), and in QEMU, the ARM draws them. The functions are the same, so a cartridge does not change.

- The ARM always transforms, lights and clips the triangles.
- The GPU fills the pixels with a 24-bit z-buffer, shades without dithering and reads textures with the nearest texel.
- If the GPU does not answer, the kernel goes back to the ARM by itself and writes it in the log.
- `stat(9)` is 1 when the GPU does the 3D.

Limits you must know

- A mesh holds up to 65535 vertices and 65535 faces (4096 and 16384 before bm3d 5.0).
- Triangles that cross the plane near the camera are cut, not dropped: floors and big objects stay whole even when they pass by the camera.
- In 256x256 games the GPU does not draw (it writes whole pages): the ARM draws.
- The GPU's z-buffer starts from zero every frame, even without `zclear()`.
- The faces that are **textured and screen door** together the GPU cannot draw: at the first one the cartridge goes to the ARM for the rest of the run (a line in the log; the mixed frame is not shown).

The order inside a frame

The waiting 3D is drawn before every 2D drawing that follows it, before `pget` and `sset`, and at the end of the frame. Draw all the 3D first and then the HUD. If a cartridge draws more 3D after the 2D in the same

frame, from the next frame (not the first) the GPU keeps the z-buffer between the two parts: about 1 MB of memory is written and read again per frame.

Tutorial: a model on the screen

You will show a model of your cartridge, lit and turning. Start from a cartridge that already has at least one model made in bm Studio (MESH section) and that opens in bm Code.

1. In `_init`, load the model: `m = model(1)`. Models count from 1. It returns `nil` if the model is not there, so check the result. Do this only in `_init`: every call to `model()` builds a new mesh.
2. In `_init`, ask for the box round the model with `bounds3d(m)`. It gives `x0, y0, z0, x1, y1, z1`. Use `(y0 + y1) / 2` to lower the model so its middle sits at height 0.
3. Place the camera once, in `_init`: `camera3d(0, 0, -6)`. The default camera is at z = -5 with a 60 degree field of view.
4. Set the light, once: `light3d(0.5, 1, -0.5, 0.3)`. The first three numbers are the light's direction and the last is the ambient light (0 to 1).
5. In `_draw`, clear the screen with `cls(0)` and then the z-buffer with `zclear()`. Do it every frame, before `draw3d`.
6. Still in `_draw`, draw the model with `draw3d(m, 0, -cy, 0, 0, ang, 0, 1, 4)`: position, then the angles `rx, ry, rz`, then the scale, then the flags. Flag 4 shades the light smoothly.
7. Add `ang = ang + 0.02` at the end of `_draw`, so the model turns a little each frame.
8. Draw the text of the game (the HUD) after `draw3d`, with `print`.
9. Run the cartridge. You should see the model lit from one side and turning. If you see nothing, the model may be behind the camera or too big. A way to find out: print the values from `bounds3d` and move the camera back.

[TO FILL: shots/api-3d-model.png]

```
local m, cy, ang = nil, 0, 0

function _init()
  m = model(1)
  local x0, y0, z0, x1, y1, z1 = bounds3d(m)
  cy = (y0 + y1) / 2
  camera3d(0, 0, -6)
  light3d(0.5, 1, -0.5, 0.3)
end

function _draw()
  cls(0)
  zclear()
  draw3d(m, 0, -cy, 0, 0, ang, 0, 1, 4)
  print("my model", 4, 4, 0xFFFFFF)
  ang = ang + 0.02
end
```

TRY IT

Change the flags from 4 to 2 and run again: the model loses its light and shows flat colours. Then try 4 + 128, which also draws the textured faces as flat colours.

The angles `rx, ry, rz` of `draw3d` are in radians. The camera's `yaw` and `pitch` are radians too; `fov` is in degrees.

Meshes and models

Function	Description
`mesh(v, f, [uv])`	a mesh from tables (up to 65535 vertices and 65535 faces: 4096 and 16384 before bm3d 5.0): `v` = {x,y,z, x,y,z, ...}, `f` = {a,b,c,colour, ...} (indices from 1; a face shows from the side where its vertices appear **clockwise**). With `uv` (6 numbers per face: u,v of the three vertices in sprite sheet pixels) the faces with colour `-1` have the sprite sheet's **texture** (perspective correct, transparent pixels stay empty). The colour can have the **material bits** (see "Drawing and material bits")
`mesh_sphere([rings, segments, c1, c2])`	a ready-made sphere. Defaults: `rings` 8, `segments` 16, `c1` 0xFFFFFF, `c2` 0xC0C0C0. There is no radius: scale it with `draw3d`
`mesh_cube([c])`	a ready-made cube. `c` defaults to 0xFFFFFF
`model(name) / model(n)`	a **3D model of the cartridge** (made with bm Studio, MESH section) as a mesh, with the sprite sheet's texture; `n` counts from 1; `nil` if it is not there. Every call builds a new mesh: do it in `_init`
`models()`	the names of the cartridge's models, in order (`{}` if it has none)
`bounds3d(m)`	`x0, y0, z0, x1, y1, z1` : the box round a mesh's vertices, in its coordinates (before moving, turning and scaling it with `draw3d`): to centre it, for collisions

Baked light

A model with **baked light** (the "lit" bit of the MESH section: the light of each corner of the faces, made by a script) is drawn smooth with that light. It has no sun or sky, but it has the lamps (`lamp3d`) and the fog. It costs less than computed light. Its textured faces (windows, signs) have one light per face, coloured, and the fog.

```
local names = models()          -- {} if the cartridge has no models
local house = model("house")    -- by name, or model(1) by number
```

Drawing and material bits

Function	Description
`draw3d(m, x, y, z, [rx, ry, rz, scale, flags])`	draws a mesh with z-buffer and per-face light. `flags` are in the table below
`zclear()`	clears the z-buffer (every frame, before `draw3d`)

The flags of draw3d

Flags add up (for example 4 + 128).

Flag	Effect
1	no z-buffer (neither test nor write): floors and backgrounds drawn first, faster
2	no light (flat colours)
4	**smooth** (Gouraud): light computed at the vertices and shaded over the face, dithered. Faces sharing the same vertex indices look like a curved surface; for sharp edges use separate vertices or the "flat" bit
8	the mesh's **shadow** on the plane `y` of the point (along the sun): it darkens what is already there, the mesh is not drawn
16, 32, 48	**level of detail** 2, 1, 0 (only that level's faces, see the material bits); without: 3, all
64	**in front** (first-person weapons and arms): the z-buffer under it is cleared, precise depths from 0.1 units
128	**flat textures**: the textured faces are drawn as flat colours, each the average of its texels. Transparent texels are skipped, and a face of holes only is grey. The mesh, bones and animation are the same; lit faces keep their light, the other faces are unchanged. On the GPU such a mesh goes triangle by triangle, a little slower. The colours are kept per mesh and texture version (bm3d 7.0)

Material bits

These bits go in a face's colour, in `mesh()` and in the models. 0 is the usual face.

Bit	Value	Effect
30	`0x40000000`	**emissive**: a flat colour, without light (lights, screens, energy)
29	`0x20000000`	**glossy**: the sun's reflection (`shine3d`)

Bit	Value	Effect
28	`0x10000000`	**screen door**: every other pixel, what is behind shows (shields, glass)
27	`0x08000000`	**flat**: also in a smooth drawing (flag 4) it takes the light of its plane; sharp edges can share vertices
24-26		**level of detail**: bits 24-25 a level `k` (0-3); bit 26 at 0 the face shows from `k` up (a detail), at 1 below `k` (a simple version)

```
local SHIELD = 0x40C8FF | 0x40000000 | 0x10000000   -- light blue, emissive, screen door
local DETAIL = 0xFFFFFF | 0x02000000                 -- white, only from detail 2 up
```

BEWARE

Faces that are both textured and screen door make the GPU give up: the cartridge goes to the ARM for the rest of the run. Use the two on different faces.

Camera, light and fog

Function	Description
`camera3d(x, y, z, [yaw, pitch, fov, roll])`	the camera (default at z = -5, fov 60 degrees); `roll` tilts the view (radians)
`light3d(x, y, z, [ambient])`	the light's direction and the ambient light (0-1)
`sky3d(sun, sky, ground)`	colours (0xRRGGBB) of the sun's light and of the ambient light coming from above and below (faces turned up take the sky, those turned down the ground); `sky3d()` goes back to white
`shine3d(spec, exponent, rim)`	the sun's reflections on glossy faces (`spec` 0-2, `exponent` 4-64: higher, smaller) and light on the shapes' rim (`rim` 0-1)
`fog3d(colour, near, far)`	fog: the faces fade into the colour between the two distances; `fog3d()` takes it away
`lamp3d(i, x, y, z, radius, [k, colour])`	point light `i` (1-4): the faces with their centre within `radius` get brighter, up to `k` more (default 1) in the middle, of the `colour` given (white if missing); `lamp3d(i)` turns it off, `lamp3d()` all. With `light3d` at a low ambient it makes dark scenes with lanterns
`shadow3d(style)`	the shadows of `draw3d` with flag 8: 0 darken (default), 1 dithered black (without reading the screen)
`textures3d([on])`	`textures3d(false)` makes every `draw3d` behave as with flag 128 (textured faces as flat colours, their average texel); `textures3d(true)` goes back to the textures (default). Returns the current state. For a "flat / textured" option in a game (bm3d 7.0)

On the GPU the shadows are always dithered black (`shadow3d` style 1), over the things already drawn thanks to the z-buffer.

```
camera3d(0, 2, -8)
light3d(0.3, 1, -0.4, 0.15)           -- low ambient: a dark scene
sky3d(0xFFF0D0, 0x6080C0, 0x302010)
fog3d(0x101820, 10, 40)
lamp3d(1, lx, ly, lz, 6, 1, 0xFFB060)  -- a lantern
```

Skeletons and animation

A model with a skeleton comes from bm Animator. The bones move the vertices while the mesh is drawn (rigid skinning), so `animate` costs only the bones.

Function	Description
`animate(m, [anim, t, anim2, t2, k, bone])`	**skeletal animation**: the model takes the pose of animation `anim` (name or number) at time `t` in seconds (looping, if the animation loops). With `anim2, t2` it blends two animations (`k` from 0, the first only, to 1, the second only: to go from one to the other). With `bone` the second holds only for that bone and those under it (a torso that shoots on running legs). Without an animation: the rest pose. Returns the animation's length. An error if the mesh has no skeleton or the animation is not there
`bone_turn(m, bone, [rx, ry, rz])`	from now on every `animate` also turns the bone by these angles (radians, x then y then z, in the parent's frame) over the animation: aiming up and down, legs that follow the walking direction. `bone_turn(m, bone)` takes it away
`bones3d(m)`	the names of the skeleton's bones, in order
`clips(m)`	a model's animations: `{ {name=, length=, loop=}, ... }` (`{}` without a skeleton)
`bone3d(m, bone)`	`x, y, z, cx, cy, cz` : where a bone's head and tail (name or number) are in the last pose, in the model's coordinates (as `bounds3d`); `nil` if the bone is not there. To attach objects to the hands (the head), a sword's tip (the tail), lights, effects
`hit3d(m, x, y, z, ry, scale, ox, oy, oz, dx, dy, dz, [maxd])`	`t, bone` : the ray from `o` along `d` against the mesh's bones in the last pose, drawn at (x, y, z) turned by `ry` and scaled. Each bone is a capsule from head to tail as wide as its vertices. The closest hit within `maxd` (`t` in units of `d`), or `nil` : hitboxes that follow the animation (headshot: bone `"head"`)

```
-- in _draw: the villager walks, then blends into a wave
local len = animate(npc, "walk", t)
animate(npc, "walk", t, "wave", t, k)      -- k from 0 (walk) to 1 (wave)
local hx, hy, hz = bone3d(npc, "hand_r")   -- a light in hand
```

[TO FILL: the real animation and bone names of a shipped model, for the example above (`"walk"` , `"wave"` , `"hand_r"` are only shapes of names)]

Effects, labels and 2D over the 3D

Function	Description
`point3d(x, y, z, radius, colour, [flags])`	a round point of that radius in the world, behind the closer things (it does not write the z-buffer): particles, sparks, bullets. `flags` 1 = every other pixel. Returns the pixels
`line3d(x0, y0, z0, x1, y1, z1, colour, [width, flags])`	a 3D line, cut by the near plane and hidden by the closer things: tracers, beams
`sprite3d(sx, sy, sw, sh, x, y, z, width, [flags])`	a rectangle of the sprite sheet facing the camera, `width` units wide in the world, hidden by the closer things: explosions, smoke, icons over the characters
`project3d(x, y, z)`	a point of the world to `sx, sy, depth` on the screen (`nil` if it is behind the camera): to draw in 2D things lined up with the 3D (horizon, sights, labels)

The GPU also draws the shadows, these effects (`point3d` , `line3d` , `sprite3d`), the screen-door faces and the textures of the models with baked light (light and fog shaded at the corners, also with the `lamp3d` s).

```
local sx, sy = project3d(ex, ey + 2, ez)    -- above an enemy
if sx then print("goblin", sx - 12, sy, 7) end
```

Visibility

Function	Description
`visible3d(x, y, z, r)`	`false` if a sphere (centre, radius) cannot be seen: behind the camera, beyond an edge of the screen, or (after `pvs3d`) on pieces of the map that the camera's cell does not see. To skip the characters and effects hidden by the walls before drawing them (bm3d 5.5)
`pvs3d(t)`	the map's precomputed visibility for `visible3d` (see below). `pvs3d()` forgets it

The table `t` of `pvs3d` is `{ x0, z0, cell, nx, nz, max_y, sets, boxes }`.

- It is a grid of `nx` by `nz` cells of side `cell` on the ground, starting from (`x0`, `z0`).
- `sets` is a string with a byte per cell (row by row along x). Each byte is the number (from 1) of a piece seen from that cell.
- `boxes` has six numbers per piece: `x0 y0 z0 x1 y1 z1`.
- With the camera above `max_y` or out of the grid only the view counts. A sphere that is off every piece is seen.

```
if visible3d(ex, ey, ez, 1.5) then
  draw3d(enemy, ex, ey, ez)
end
```

The GPU: gpu3d and its options

`gpu3d([on, aa, vs, queue, impostor, later])` returns `on, aa, vs, version, queue, impostor, later`. With arguments it changes the choice for this cartridge (for example a "3D: GPU / GPU + VS / GPU + AA / ARM" menu in the game). At the end of the run it goes back to the settings'.

Argument	Meaning
`on`	whether the GPU draws the 3D. Stays `false` if there is no GPU (QEMU, `gpu3d=0`) and in 256x256 games (the GPU writes whole pages)
`aa`	whether with 4x MSAA. Stays `false` if MSAA cannot be used
`vs`	whether the GPU's vertex shader places the models' vertices: `false`, `1` (the scenery: models without light or with light at the corners) or `2` (all, also those lit by the sun and with bones; `true` is 2). Stays `false` if the boot test did not see the vertex shader (and its clipping) work
`queue`	whether the GPU's job starts without waiting for it and the next frame's `_update` runs meanwhile (where the boot test saw it work). The 2D drawn after the 3D (the HUD) and that of the `_update` are recorded and go on the page when the GPU is done, in the same order: the result is the same. An `_update` that draws 3D or reads the screen with `pget` goes back to running after the frame. `2`: two jobs in flight. `true` asks for the console's own level (`gpu3d_queue` in Settings, 1 or 2); `false` for none. The fifth value returned is the level in force (`1`, `2`) or `false`
`impostor`	off by default. A model whose box on the screen is at most that many pixels (2-64) of a 1080-line screen is drawn as one quad facing the camera in its average colour, lit as a whole (2 triangles instead of all its faces), by the GPU or the ARM alike. `false` or `0` turns it off
`later`	the 6th argument, only with `queue 2`; never in a game played `online`. Each frame is shown a frame later, so the next frame's `_update` and `_draw` run while the GPU draws it: the same pictures, a frame later (see "A frame later")

The value `version` is the version of the 3D drivers this choice reproduces: `"0.2"` the ARM, `"2.1"`, `"3.0"`, `"3.4"`, `"4.1"`, `"6.9"` a frame later (`docs/DRIVERS.md`).

Impostors

- The game's screen is scaled to the TV's height. At 640x360 the limit is a third as many of the game's pixels, at 720p two thirds, so a model looks the same size when it is chosen whatever the resolution.
- A model with bones too is drawn as an impostor, by the box of its pose (where `animate()` put each bone's part). An animated model out of the view is then also skipped, as one without bones.

- It can be given with `on` `nil` : `gpu3d(nil, nil, nil, nil, 16)` .
- The choice depends only on the camera, the model, its pose and the size of the game's screen, so it is the same on every console of a match.
- The console's default is `r3d_impostor` in `bm/config.txt` (*Settings* > *Screen and sound* > *3D impostors*).

What the GPU also does

The GPU draws the shadows (`draw3d` with flag 8), the effects, the screen-door faces and the textures of the models with baked light. A frame that starts with `cls()` costs the GPU less: the tiles start from the colour of `cls` instead of reading the page again. Textures with sides that are multiples of 32 (128x128 sprite sheets, 256x256, ...) go to the GPU in T-format, the tiled format of its cache, faster to read.

Anti-aliasing

Settings > *Graphics* > *3D anti-aliasing: 4x* (`gpu3d_aa=1` in `bm/config.txt`) makes the GPU draw the 3D with 4x MSAA: four samples per pixel, the average at the end of the tile, triangles' edges without steps. Nothing changes in the cartridges.

- It holds only where the GPU bears it (the boot test says *4x: not on this GPU* otherwise).
- It holds only in the jobs without a kept z-buffer: cartridges that draw 3D, then 2D, then more 3D in the same frame stay without anti-aliasing (the 4-sample z-buffer cannot be saved).
- If the test finds the GPU cannot load the page again into the 4 samples, MSAA is used only in the frames that start with `cls()` .
- On the ARM there is no anti-aliasing.

A frame later

Settings > *Screen and sound* > *3D frame shown: A frame later* (`gpu3d_later=1` , with the frame queue on two jobs, `gpu3d_queue=2` ; from a game the sixth argument of `gpu3d`) shows each frame a frame later. At the end of `_draw` the GPU starts the frame's 3D and the frame waits, while the next `_update` and `_draw` run on another page. The 2D drawn after the 3D goes on the waiting frame when the GPU is done, and that frame is shown at the end of the next one.

- Nothing changes in the cartridges and the pictures are the same, a frame later: the input reaches the screen a frame later than without the queue.
- Keep the game's state in `_update` (as for online games) and draw the HUD after the 3D. `pget` reads the page being drawn.
- It is not used in games played `online` (left out there), via RAM (`light_begin` , `dark_begin`) and in the frame of a `screen()` change.
- It is off by default until it is measured on the Pi.

Collision worlds

Solid boxes, rays and bodies that move sliding on the walls, in C: much faster than in Lua. Complete example: `carts/overbit`. For the 2D map there are the tiles' flags (`mflags`) and bmlib's `lib.move` / `lib.step` .

Function	Description
`world3d()`	an empty collision world
`world_box(w, x0, y0, z0, x1, y1, z1, [tag])`	a solid box; returns its number
`world_ray(w, ox, oy, oz, dx, dy, dz, [maxd, ground])`	`t, nx, ny, nz, box` : the first point hit along the ray (the face's normal; box 0 = the ground `y = 0` , which counts unless `ground` is `false`), or `nil`
`world_move(w, x, y, z, r, h, dx, dy, dz, [step, on_ground])`	`x, y, z, flags` : a body (feet at `x, y, z` , radius `r` , height `h`) moved by `d` , sliding on the walls and climbing steps up to `step` (0.45) if it was on the ground. `flags` : 1 on the ground, 2 a wall (4 along x, 8 along z), 16 a ceiling
`world_floor(w, x, z, y, r, [step])`	the floor's height under `(x, z)`

```
local w = world3d()
world_box(w, -10, -1, -10, 10, 0, 10)          -- a slab
local x, y, z, fl = world_move(w, px, py, pz, 0.4, 1.8, vx, vy, vz, 0.45, on_ground)
on_ground = (fl & 1) ~= 0
```

If `on_ground` is missing in `world_move` it counts as `false` . The `tag` of `world_box` defaults to 0.

[TO FILL: what the `tag` of `world_box` is used for]

Light for 2D scenes

These functions are not 3D: they darken a 2D scene lit by lamps. From the first `light_begin()` or `dark_begin()` the cartridge draws in RAM instead of directly on the screen.

Light

Dark scenes lit only by lamps, candles, torches: the frame's drawing is multiplied by a "light map" computed in C (a grid every 4 pixels, interpolated and slightly dithered).

Function	Description
`light_begin([ambient])`	starts the frame's lights: `ambient` is the colour of the background light (`0x000000` pitch dark, `0xFFFFFF` no effect)
`light(x, y, radius, colour, [intensity])`	a soft light in world coordinates (`camera` holds); more lights add up, up to 2x the brightness
`light_end()`	applies the light to everything drawn; what you draw after (HUD, text) stays at full light

```
cls(0); map(...); spr(...)                  -- the scene
light_begin(0x0A0A16)                        -- dark blue night
light(lx, ly, 50 * (0.95 + math.random() * 0.1), 0xFFB060)   -- a flickering street lamp
light(px, py, 40, 0xFFC888, 0.9)             -- the player's lantern
light_end()
print("life", 4, 4, 0xFFFFFF)                -- the HUD is not darkened
```

Complete example: `carts/hunt` .

Light by levels

The other light works by levels: each pixel has a **light level** (0 the darkest) and its colour becomes the one a **fade table** gives at that level. Lamps make concentric rings of levels, from their level in the middle down to 0 at the edge. The rings' edges are mixed with a 4x4 ordered dither; where two lamps meet the stronger wins. The tables are chosen by the cartridge: shadows can turn night blue and colours near the lamps orange, staying on the colours of its own palette. All in C.

Function	Description
`fades(tables)`	the tables: `{ {colour, l0, l1, ...}, ... }` , for each colour of the palette what it becomes at level 0 (the darkest), 1, ...; all rows have the same number of levels (2-16, up to 255 colours). The colours without a table are scaled as the average of the tables. Returns the number of levels
`dark_begin([ambient])`	starts the frame: every pixel at level `ambient` (default 0); from here the cartridge draws in RAM
`glow(x, y, radius, level, [dither])`	a lamp in world coordinates (`camera` holds): `level` in the middle, 0 at `radius` ; `dither` 0-1 (default 0.5) is how much the rings' edges mix (0 sharp rings, 1 a continuous dithered shade)

Function	Description
`dark_end()`	applies the levels to everything drawn; what you draw after (flames, sparks, HUD) stays as it is and "shines"

```
fades(TABLES)                              -- once, in _init
cls(0); map(...); spr(...)                 -- the scene at full light
dark_begin(1)                              -- night: level 1 everywhere
glow(lx, ly, 72 + math.random(2), 6)       -- a street lamp
glow(px, py, 28, 3)                        -- the little light round the player
dark_end()
spr(FLAME, fx, fy)                         -- the flames are not darkened
```

Complete example: `carts/yharnam`. A small one: `SQUARE_CART` in `tests/qemu_test.py` (a lamp on a 256x256 cartridge).

The n8 library

Every cartridge also sees the `n8` table: the **nano8** machine, with the functions of the `.p8` cartridges (`n8.spr`, `n8.map`, `n8.print`, `n8.peek` and the others, with their arguments and numbers as they want them) and those to load, start and show them (`n8.load`, `n8.power`, `n8.buttons`, `n8.blit`, `n8.preview`, `n8.compile`). It is made for `carts/nano8`; a `.bm` game does not need it.

Performance tips

These are the tips the repository states; the numbers come from the 3D Bench (*Dev > 3D Bench*, or the `j` key of the monitor).

- Draw all the 3D first, then the 2D and the HUD. 3D after 2D in the same frame makes the GPU keep the z-buffer (about 1 MB written and read again per frame).
- Start the frame with `cls()` : the GPU tiles start from its colour and do not read the page again.
- Draw floors and backgrounds first with flag 1 (no z-buffer): faster.
- Use models with baked light for scenery: they cost less than computed light.
- Skip what cannot be seen with `visible3d` (and `pvs3d` for maps) before drawing it.
- Use `impostor` for crowds: a far model becomes one quad (2 triangles).
- Build models in `_init` , not in `_draw` : every `model()` call builds a new mesh.
- Use textures whose sides are multiples of 32 (128x128, 256x256): the GPU reads them faster.
- Use the collision worlds instead of the same tests in Lua: they are in C and much faster.
- One measured scene (640x360 EXTREME) had a GPU job of 4.2 ms in a 19.6 ms frame: the ARM was setting the pace. At 1080p a played match is fill-bound: fewer pixels, not fewer triangles, is the lever. With `stat(16)` close to the frame time the GPU sets the pace; well below it, the ARM does.
- Measure: `stat(16)` is the GPU's job time of the last frame in ms (0 on the ARM). With the queue on, it includes the wait before the ARM looks: trust it only without the queue.
- The bench scores each driver against bm3d 2.1 on the Pi Zero W (1000 points). `make test-b3d` runs a short check on the PC; the PC's milliseconds do not count.

> **NOTE**
> The 3D Bench measures each test with the ARM and with every GPU option that the console can reproduce, so you can compare the profiles.

[TO FILL: figures in frames per second for a typical cartridge on the Pi Zero W and on the RGB30, measured with the 3D Bench]

Open points

- **Tutorial: a model on the screen**: picture to take: shots/api-3d-model.png
- **Skeletons and animation**: the real animation and bone names of a shipped model, for the example above (`"walk"` , `"wave"` , `"hand_r"` are only shapes of names)
- **Collision worlds**: what the `tag` of `world_box` is used for
- **Performance tips**: figures in frames per second for a typical cartridge on the Pi Zero W and on the RGB30, measured with the 3D Bench